l

INSTITUTO TECNOLÓGICO DE MÉXICO

CAMPUS ORIZABA

“ACTIVIDAD 2”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 4 DE MARZO DE 2026

Problema 1:

En cierta empresa de fabricación de ropa, se requiere sistematizar el proceso, se fabrican diversidad de prendas de vestir tanto para caballeros como damas, no se fabrica ropa de niños, de cada prenda se tiene un registro de modelo, tela, costo de producción por pieza, genero (masculino, femenino o mixto), temporada (primavera, verano, otoño o invierno), cuando se produce un lote de cada prenda, se registra número de lote, número de piezas, fecha de fabricación. De cada lote debe calcularse el costo de producción del lote, así como el monto de recuperación que se obtendrá después de su venta, el precio de venta por pieza es siempre 15% más del costo de producción y por lote completo es por cada pieza el 5% más del costo de producción.

Análisis Del Problema:

En este problema vemos que es sobre una fábrica de ropa, lo cual lo dividimos en dos clases:

Clase Prenda: La clase prenda es para cada tipo de prenda que se fabrica, ya que cuenta con modelo, tela utilizada, costo de producción por pieza, género y la temporada en la que se fabricó. Una prenda entonces es única ya que cada una tiene su propio lote y por lo cual son independientes entre si

La prenda permite permite calcular el precio de venta, lo cual siempre es un 15% mayor al costo de producción

Clase Lote: La clase lote fue creada para que cada lote tenga su número de lote, fecha de fabricación y el número de piezas producidas. A partir del lote se puede calcular el costo total de producción y el monto de recuperación después de la venta Estadística

Clase prenda:

Prenda			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Modelo	String	1	si
Tela	String	1	si
CostoProduccionPZ	Double	1	si
Genero	String	1	si
Temporada	String	1	si

Operaciones:

Prenda			
operación	Tipo de dato(retorno)	argumentos	Alcance
PreciosVentaPieza		Double	Public

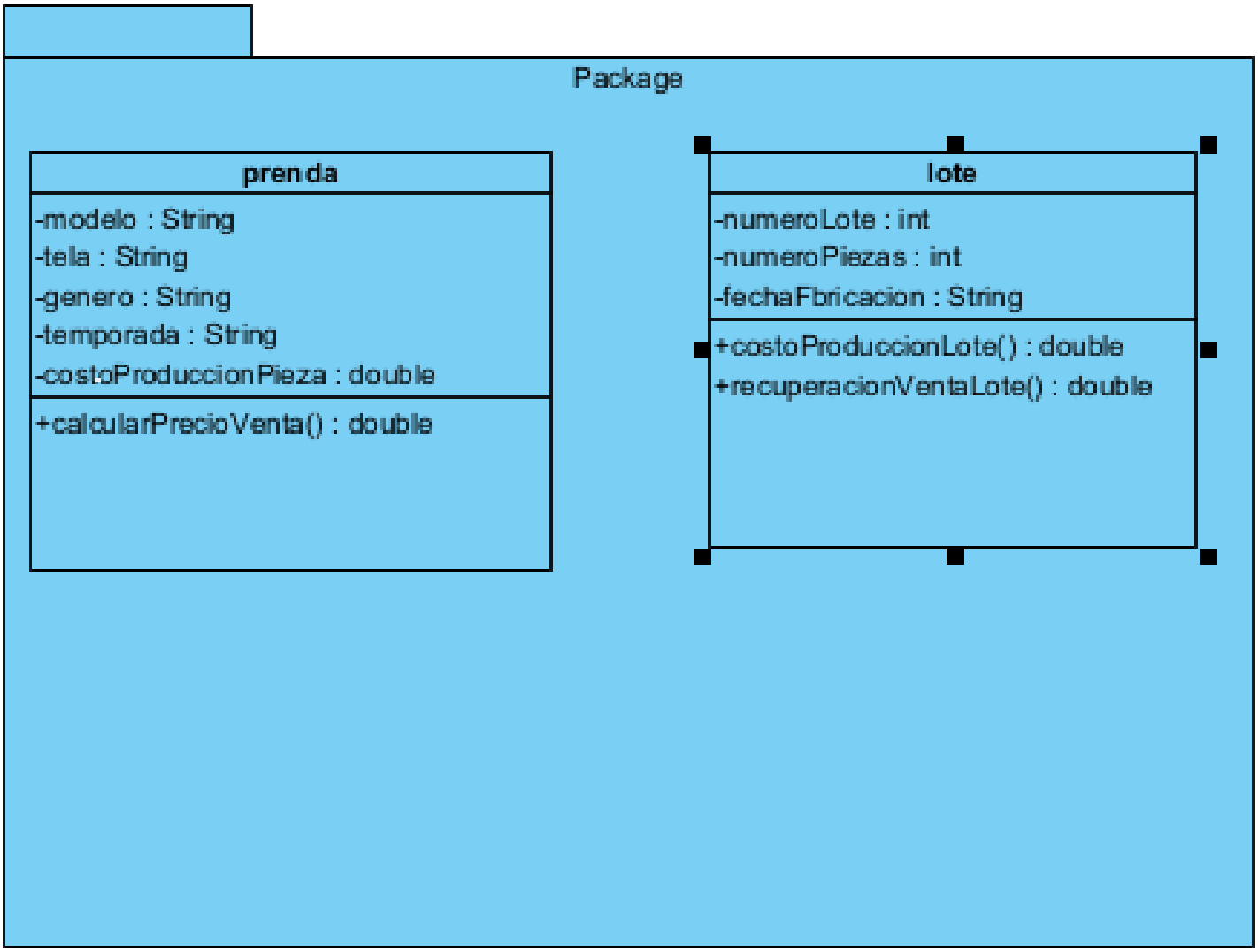
Clase lote:

Lote			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Numero de lote	Int	1	Si
Número de piezas	int	1	Si
Fecha de fabricación	LocalDate	1	Si

Operaciones:

Lote			
operación	Tipo de dato(retorno)	argumentos	Alcance
CostoProduccionLote	float		public
CalcularMontoRecuperacionLote	float		public

Diagrama:


Package
prenda
-modelo : String
-tela : String
-genero : String
-temporada : String
-costoProduccionPieza : double
+calcularPrecioVenta() : double
lote
-numeroLote : int
-numeroPiezas : int
-fechaFbricacion : String
+costoProduccionLote() : double
+recuperacionVentaLote() : double

Código clase prenda:

```java
prenda.java ×   lote.java

public class prenda {  no usages  new *
    private String Tela;  2 usages
    private double precioventaPZ;  3 usages
    private String genero;  2 usages
    private String temporada;  2 usages

    public prenda(String modelo, String tela, double precioventaPZ, String genero, String temporada) {  no usages  new *
        this.modelo = modelo;
        Tela = tela;
        this.precioventaPZ = precioventaPZ;
        this.genero = genero;
        this.temporada = temporada;
    }

    public double precioventaPZ() {  no usages  new *
        return precioventaPZ*1.15;
    }

    @Override  new *
    public String toString() {
        return "prenda{" +
                "modelo='" + modelo + '\'' +
                ", Tela='" + Tela + '\'' +
                ", precioventaPZ=" + precioventaPZ +
                ", genero='" + genero + '\'' +
                ", temporada='" + temporada + '\'' +
                '}';
```

Prueba clase prenda:

```
jshell> double precioFinal=miropa.precioventaPZ();
precioFinal ==> 114.99999999999999

jshell> miropa.toString();
$4 ==> "prenda{modelo='pantalon', Tela='mezclilla', precioventaPZ=100.0, genero='M', temporada='verano'}"
```

```
jshell> import fabrica.ropa.prenda;

jshell> prenda miropa= new prenda ("pantalon","mezclilla", 100.00, "M", "verano");
miropa ==> prenda{modelo='pantalon', Tela='mezclilla', preci ... o='M', temporada='verano'}
```

Análisis del código: Aquí, lo que hicimos fue hacer la variable miprenda que recibirá todos los datos que nosotros escribamos de acuerdo a los atributos que nosotros declaramos en la clase prenda, y asignamos y escribimos cada valor requerido, vemos que si nos imprimió nuestra new prenda "miropa". Si nos da lo que pide el problema que es acceder de cada prenda, que tipo de prenda es, su tipo de tela, precio por pieza, género y temporada.

Código Clase Lote:

```java
prenda.java    lote.java ×

public class lote {  no usages  new *
    private int numDeLote;  2 usages
    private int numDePIezas;  4 usages
    private LocalDate fechadeFabricacion;  2 usages
    private double costoProduccionPz;  3 usages

    public lote(int numDeLote, int numDePIezas, LocalDate fechadeFabricacion, double costoProduccionPz) {  no usages  new *
        this.numDeLote = numDeLote;
        this.numDePIezas = numDePIezas;
        this.fechadeFabricacion = fechadeFabricacion;
        this.costoProduccionPz = costoProduccionPz;
    }

    public double costoProduccionLote(){  no usages  new *
        return numDePIezas*costoProduccionPz;
    }
    public double calcularMontoRecuperacion(){  no usages  new *
        double precioVenta=costoProduccionPz*1.15;
        return numDePIezas*precioVenta;
    }

    @Override  new *
    public String toString() {
        return "lote{" +
                "numDeLote=" + numDeLote +
                ", numDePIezas=" + numDePIezas +
                ", fechadeFabricacion=" + fechadeFabricacion +
                '}';
```

Prueba código clase Lote:

```
jshell> import fabrica.ropa.lote;

jshell> import java.time.LocalDate;

jshell> lote miLote = new lote(101, 50, LocalDate.now(), 120.50);
miLote ==> lote{numDeLote=101, numDePIezas=50, fechadeFabricacion=2026-03-02}

jshell> double totalProduccion = miLote.costoProduccionLote();
totalProduccion ==> 6025.0

jshell> double recuperacion = miLote.calcularMontoRecuperacion();
recuperacion ==> 6928.749999999999
```

```
jshell> miLote.toString();
$10 ==> "lote{numDeLote=101, numDePIezas=50, fechadeFabricacion=2026-03-02}"
```

Análisis del código:

Aquí hacemos lo mismo y vemos que si nos retorna cada valor, primero, creamos milote, y asignamos cada valor requerido dependiendo lo declarado en la clase, y vemos que si nos da la información que nos solicitaba el problema, de cada lote se necesitaba el número de lote, número de piezas del lote y fecha de fabricación del lote.

También cumple con la operación de obtener el costo de producción por lote, que se realizó multiplicando el número de piezas por el costo de producción por pieza, y con la operación de calcular el monto de recuperación, cumpliendo con lo que pedía el problema, aumentar el 15% al costo de producción, lo que hicimos fue calcular el costo venta multiplicando el costo de producción por el 15%, después multiplicamos el número de piezas por el precio de venta.

Problema 2:

Una empresa agrícola ha decidido sistematizar el control de la producción frutal que tiene, para lo cual de cada fruta se describe su nombre, la extensión del terreno (hectáreas) dedicadas a su cultivo y producción, el tiempo de cosecha (Cada fruta puede tener 1 o más periodos de cosecha), la cantidad estimada a cosechar por hectárea en cada periodo (cada periodo por condiciones de estación climática puede tener diferentes cantidades estimadas), costo de producción promedio por tonelada y el precio de venta promedio por tonelada. Por cada fruta se puede agregar o eliminar un periodo de cosecha, se puede obtener el costo total de un periodo, así como las ganancias estimadas en un periodo.

Análisis Del Problema:

Ahora para este problema fue una Empresa que se dedica a vender frutas y ocupamos dos clases que son:

Clase Fruta: Que define la extensión del cultivo. Como atributos pusimos lo que fue el Nombre de tipo String para nombrar cada fruta, extensión del terreno como tipo de dato Double debido a que las hectáreas son mayormente en unidad decimal, el Precio Venta en Toneladas y Precio Venta en Toneladas también se ocupó de tipo float para saber exactamente cuánto nos cuesta cada tonelada y en cuanto se puede vender

Su Operación fue Agregar el Periodo y Eliminar el Periodo de su cosecha

Clase Periodo Cosecha: Ocupamos el NombrePeriodo de tipo String para asignarle un nombre al periodo y la CantidadToneladas de tipo Double ya que queremos saber la cantidad que se puede esperar y sea “exacta”.

Sus operaciones son ProduccionTotal con el tipo de dato de retorno Double para poder saber la producción por hectáreas. Otra de sus operaciones es CostoProduccion, con tipo de dato de retorno Double, ya que el problema nos pide un costo de producción promedio por tonelada, también otra operacion dentro de la clase es GananciaTotal, que retornara el tipo de dato Double, ya que el problema también nos comenta que se puede obtener las ganancias estimadas de un periodo.

Clases Fruta

fruta			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Nombre fruta	String	1	Si
Hectáreas	Double	1	Si
CostoProducciónToneladas	Double	1	Si
PrecioVentaToneladas	Double	1	Si

Operaciones:

Fruta			
Operación	Tipo de dato (retorno)	Argumentos	Alcance
Agregar Periodo	void	Periodo, periodo de cosecha	public
Eliminar Periodo	void	Periodo, periodo de cosecha	public

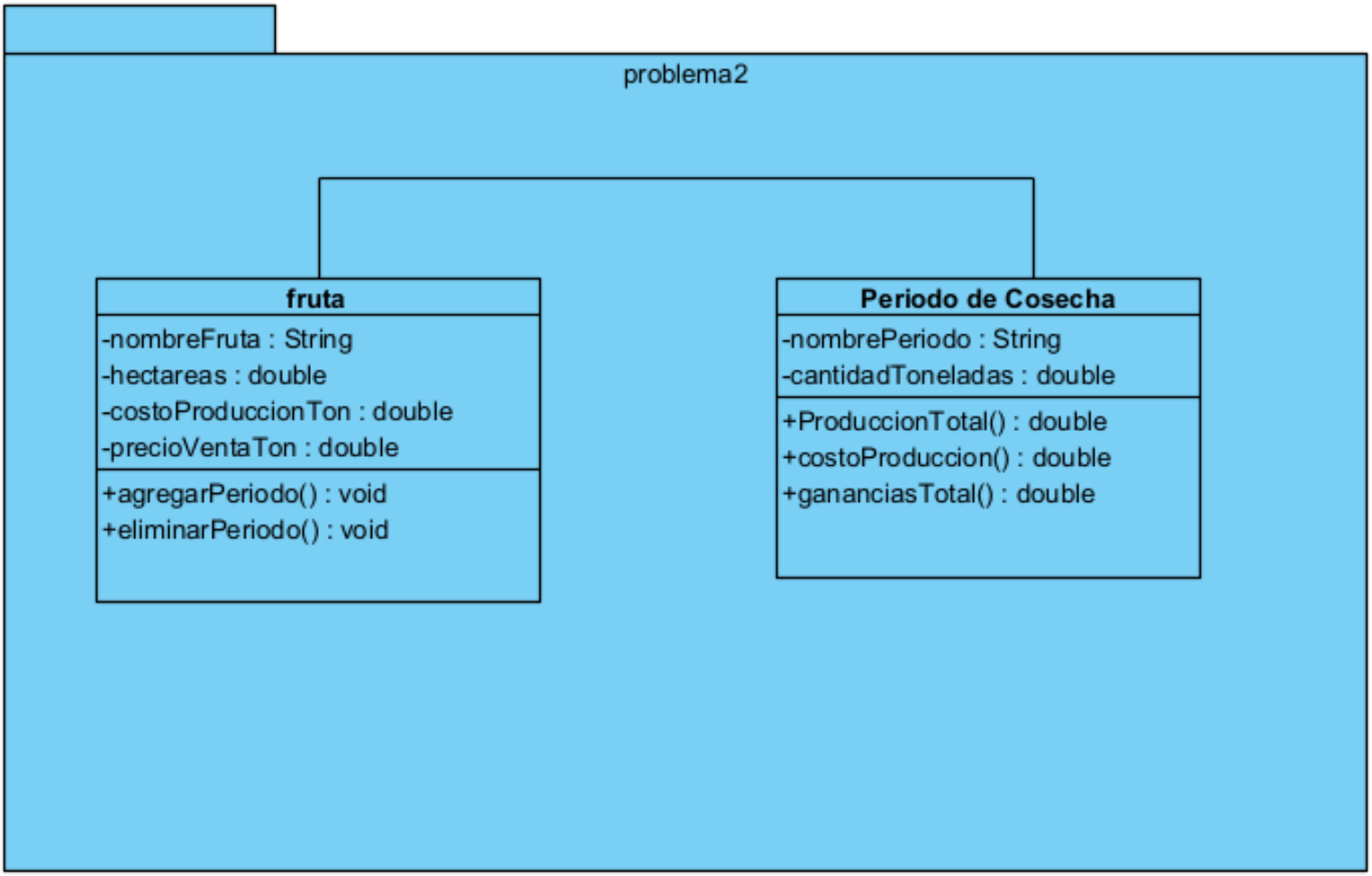
Clase Periodo Cosecha:

PeriodoCosecha			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
NombrePeriodo	String	1	Si
CantidadToneladas	Double	1	Si

Operaciones:

periodoCosecha			
Operación	Tipo de dato (retorno)	argumentos	Alcance
ProduccionTotal	Double	Hectáreas	Publico
CostoProduccion	Double	Hectáreas y costo toneladas	Publico
GananciaTotal	Double	Hectáreas, costo toneladas y precio a vender por tonelada	Publico

Diagrama:


problema2
fruta
-nombreFruta : String
-hectareas : double
-costoProduccionTon : double
-precioVentaTon : double
+agregarPeriodo() : void
+eliminarPeriodo() : void
Periodo de Cosecha
-nombrePeriodo : String
-cantidadToneladas : double
+ProduccionTotal() : double
+costoProduccion() : double
+gananciasTotal() : double

Código Clase Fruta:

fruta.java × periodoCosecha.java

```java
package empresa.agricola;
import java.util.ArrayList;


public class fruta {  no usages  new *
        private String nombreFruta;  2 usages
        private double hectareas;  2 usages
        private double costoProduccionTon;  2 usages
        private double precioVentaTon;  2 usages

        private ArrayList<periodoCosecha> periodos;  3 usages

        public fruta(String nombreFruta, double hectareas, double costoProduccionTon, double precioVentaTon){  no usages  new *
            this.nombreFruta = nombreFruta;
            this.hectareas = hectareas;
            this.costoProduccionTon = costoProduccionTon;
            this.precioVentaTon = precioVentaTon;
            this.periodos = new ArrayList<>();
        }

        public void agregarPeriodo(periodoCosecha periodo){  no usages  new *
            periodos.add(periodo);
        }
```

fruta.java × periodoCosecha.java

```java
public class fruta {  no usages  new *

        public void agregarPeriodo(periodoCosecha periodo){  no usages  new *
            periodos.add(periodo);
        }

        public void eliminarPeriodo(periodoCosecha periodo){  no usages  new *
            periodos.remove(periodo);
        }

        @Override  new *
        public String toString(){
            return "Fruta: " + nombreFruta +
                    "\nHectareas: " + hectareas +
                    "\nCosto Produccion Ton: " + costoProduccionTon +
                    "\nPrecio Venta Ton: " + precioVentaTon;
        }
}
```

Código Clase PeriodoCosecha

fruta.java periodoCosecha.java

```java
package empresa.agricola;

public class periodoCosecha {  no usages  new *
    private String NombrePeriodo;  2 usages
    private double CantidadToneladas;  3 usages


    public periodoCosecha(String nombrePeriodo, double cantidadToneladas) {  no usages  new *
        NombrePeriodo = nombrePeriodo;
        CantidadToneladas = cantidadToneladas;
    }

    public double ProduccionTotal(double Hectareas) {  2 usages  new *
        return CantidadToneladas*Hectareas;

    }
    public double CostoProduccion(double Hectareas, double CostoTonleadas){  no usages  new *
        return ProduccionTotal(Hectareas)*CostoTonleadas;
    }

    public double gananciaTotal(double Hectareas,double CostoToneladas, double precioVenta){  no usages  new *
        double produccion=ProduccionTotal(Hectareas);
        return (produccion*precioVenta)- (produccion*CostoToneladas);
    }
```

fruta.java periodoCosecha.java

```java
public class periodoCosecha {  no usages  new *
        return ProduccionTotal(Hectareas)*CostoTonleadas;
    }

    public double gananciaTotal(double Hectareas,double CostoToneladas, double precioVenta){  no usages  new *
        double produccion=ProduccionTotal(Hectareas);
        return (produccion*precioVenta)- (produccion*CostoToneladas);
    }

    @Override  new *
    public String toString() {
        return "PeriodoCosecha{" +
                "NombrePeriodo='" + NombrePeriodo + '\'' +
                ", CantidadToneladas=" + CantidadToneladas +
                '}';
    }
}
```

Prueba Clase Fruta:

```
jshell> import empresa.agricola.fruta;

jshell> fruta fruta1= new fruta ("mango", 20.8,40.0, 1000.0);
fruta1 ==> Fruta: mango
Hectareas: 20.8
Costo Produccion Ton: 40.0
Precio Venta Ton: 1000.0

jshell> fruta1
fruta1 ==> Fruta: mango
Hectareas: 20.8
Costo Produccion Ton: 40.0
Precio Venta Ton: 1000.0
```

Prueba Clase PeriodoCosecha:

```
jshell> import empresa.agricola.periodoCosecha;

jshell> periodoCosecha p=new periodoCosecha("primavera", 50.0);
p ==> PeriodoCosecha{NombrePeriodo='primavera', CantidadToneladas=50.0}

jshell> fruta1.agregarPeriodo(p);

jshell> p.ProduccionTotal(50);
$7 ==> 2500.0

jshell> p.gananciaTotal(20,40,1000);
$8 ==> 960000.0
```

Resultados métodos:

```
jshell> fruta1
fruta1 ==> Fruta: mango
Hectareas: 20.8
Costo Produccion Ton: 40.0
Precio Venta Ton: 1000.0

jshell> p
p ==> PeriodoCosecha{NombrePeriodo='primavera', CantidadToneladas=50.0}
```

Análisis del código:

Primero vemos que si nos deja acceder a la información de la clase fruta, nos muestra dentro de la variable asignada llamada fruta1, todo lo requerido, que es el tipo de fruta, hectáreas, costo de producción y precio venta por toneladas. Vemos que también cumple con poderle agregar a la clase fruta este periodo "p"

Después vemos la clase periodoCosecha que si nos imprime nuestros datos que nos pedía "Nombre del periodo" y "Cantidad Toneladas". El problema nos pedía poder calcular la producción total por hectáreas, lo que hicimos aquí fue multiplicar la cantidad toneladas por hectáreas. También nos pedía calcular el costo producción con respecto a las hectáreas, aquí multiplicamos la producción total por el costo con respecto a las toneladas, por último, nos pide calcular la ganancia Total, este fue un poco más complicado porque tenía más argumentos que los demás, pero primero hacemos una variable que va a tener como valor un double llamada producción, esta será igual a producción total por hectáreas, porque lo que necesitamos para calcular nuestras ganancias totales es saber cuánto hicimos en total, entonces multiplicamos nuestra producción por el precio venta y después multiplicamos la producción por el costo por toneladas y restamos el resultado de estas dos multiplicaciones.

Problema 3:

El observatorio astronómico de San Pedro Mártir, de acuerdo a sus observaciones necesita un catálogo de los cuerpos celestes descubiertos para lo cual de cada cuerpo celeste se describe la ubicación en el espacio estelar en relación a las posiciones geográficas espaciales (expresado en grado de latitud (norte o sur) y grados de longitud (este o oeste)) periodo de ubicación (mes(es) donde se observó en dicha posición), distancia aproximada a la tierra (años luz o km/segundos), el nombre que se le asigno, la composición del cuerpo estelar (gases, solidos, líquidos). De cada cuerpo celeste se puede obtener el desplazamiento con respecto a otra observación del mismo cuerpo en un periodo distinto.

Análisis del problema:

Clase cuerpo celeste: La clase cuerpo celeste se creó para poder identificar los cuerpos celestes por su nombre, los materiales que lo conforman y la distancia estimada a la tierra.

Clase periodo visible: el problema nos indica que la distancia se calcula comparando los dos puntos diferentes en los que se encuentra o encontraba el cuerpo celeste en diferentes etapas (mes/meses).

Clase Ubicación espacial: para poder calcular la distancia, en el problema también nos indica que el cuerpo celeste en el momento que se registra se obtienen ciertas características del cómo la latitud y longitud de este.

Clases Cuerpos celestes

Cuerpos celestes			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Nombre	String	1	si
Composición	Double	1	si
DistanciaTierra	Double	1	si
UnidadTierra	Double	1	si

Operaciones:

Cuerpos celestes			
Operación	Tipo de dato (retorno)	argumentos	Alcance
CalcularDesplazamiento	Double	Dos observaciones, una primero y otra después	public

Clase periodo Visible:

Periodo Visible			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Periodo	String	1	si

Operaciones:

Periodo Visible			
Operación	Tipo de dato (retorno)	argumentos	Alcance
Ubicacion	Double		public

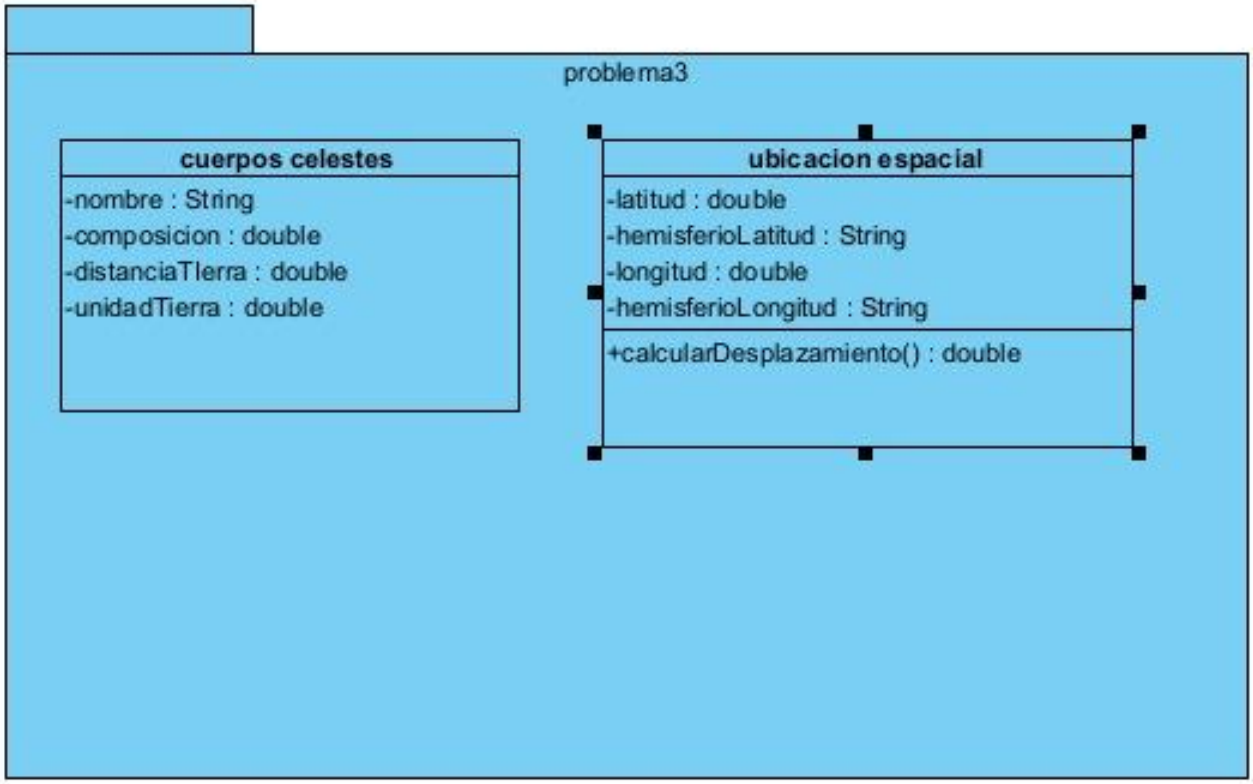
Clase Ubicación Especial

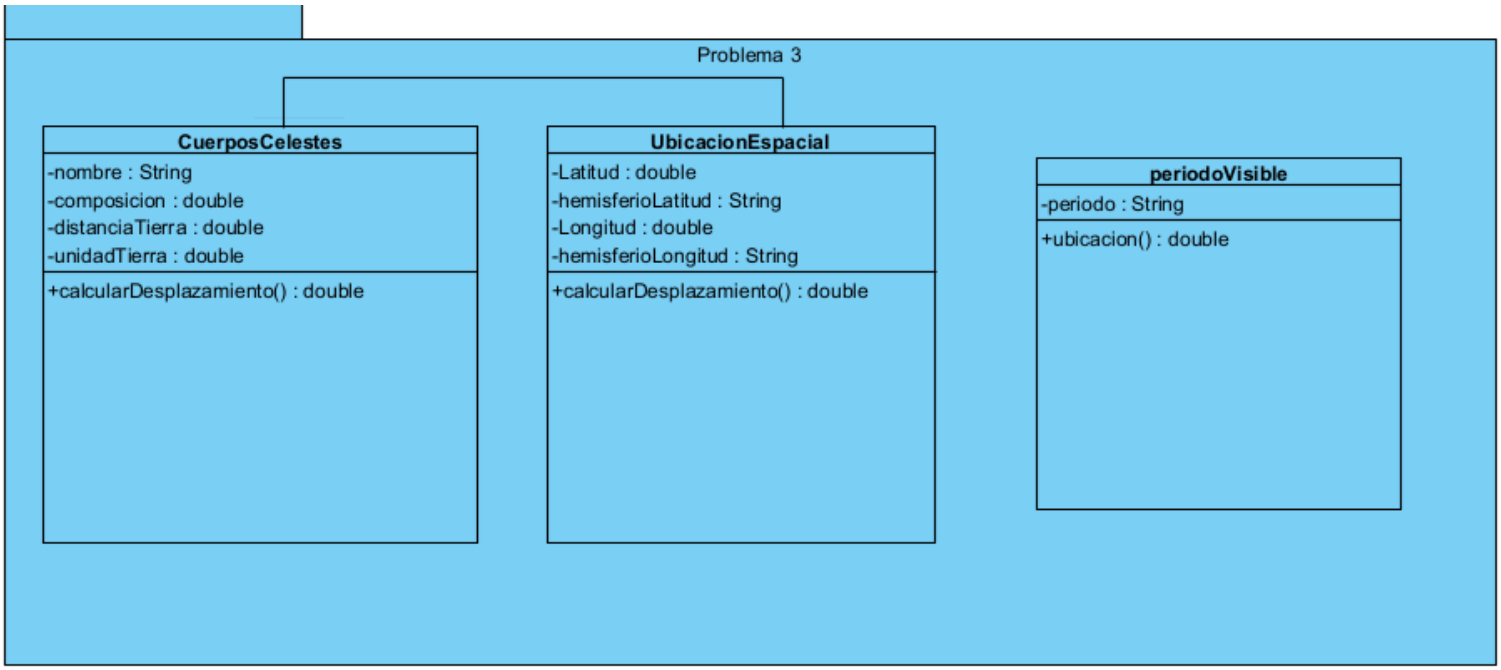
Ubicación Espacial			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
Latitud	Double	1	Si
HemisferioLatitud	String	1	Si
Longitud	Double	1	Si
HemisferioLongitud	String	1	Si

Operaciones:

Ubicación Espacial			
Operación	Tipo de dato (retorno)	argumentos	Alcance
CalcularDesplazamiento	Double	Otra ubicación y la ubicación ya obtenida del mismo	Public

Diagrama:


problema3
cuerpos celestes
-nombre : String
-composicion : double
-distanciaTIerra : double
-unidadTierra : double
ubicacion espacial
-latitud : double
-hemisferioLatitud : String
-longitud : double
-hemisferioLongitud : String
+calcularDesplazamiento() : double


Problema 3
CuerposCelestes
-nombre : String
-composicion : double
-distanciaTierra : double
-unidadTierra : double
+calcularDesplazamiento() : double
UbicacionEspacial
-Latitud : double
-hemisferioLatitud : String
-Longitud : double
-hemisferioLongitud : String
+calcularDesplazamiento() : double
periodoVisible
-periodo : String
+ubicacion() : double

Código Clase CuerposCelestes:

```java
package observatorio;
import java.util.ArrayList;
public class cuerposCelestes {  no usages  Santiago Rojas Bonilla


        private String nombre;  2 usages
        private double composicion;  2 usages
        private double distanciaTierra;  2 usages
        private double unidadTierra;  2 usages
        private ArrayList <PeriodoVisible> periodos;  1 usage

        public cuerposCelestes(String nombre, double composicion, double distanciaTierra, double unidadTierra, ArrayList<PeriodoVisible>
            this.nombre = nombre;
            this.composicion = composicion;
            this.distanciaTierra = distanciaTierra;
            this.unidadTierra = unidadTierra;
            this.periodos = new ArrayList<>();
        }

        @Override  Santiago Rojas Bonilla
        public String toString() {
            return "cuerposCelestes{" +
                    "nombre='" + nombre + '\'' +
                    ", composicion=" + composicion +
                    ", distanciaTierra=" + distanciaTierra +
```

Prueba clase CuerposCelestes:

```
jshell> import observatorio.cuerposCelestes;


jshell> cuerposCelestes marte = new cuerposCelestes("Marte",0.15,225.0,1.0);
marte ==> cuerposCelestes{nombre='Marte', composicion=0.15, ... a=225.0, unidadTierra=1.0}

jshell> toString();
|  Error:
|  non-static method toString() cannot be referenced from a static context
|  toString();
|  ^------^

jshell> System.out.println(marte);
cuerposCelestes{nombre='Marte', composicion=0.15, distanciaTierra=225.0, unidadTierra=1.0}
```

Código Clase UbicacionEspacial:

```java
package observatorio;

public class ubicacionEspacial {  3 usages  Santiago Rojas Bonilla
    private double latitud;  3 usages
    private String hemisferioLatitud;  1 usage
    private double longitud;  3 usages
    private String hemisferioLongitud;  1 usage

    public ubicacionEspacial(double latitud, String hemisferioLatitud, double longitud, String hemisferioLongitud) {  no usages  Santiago Roja
        this.latitud = latitud;
        this.hemisferioLatitud = hemisferioLatitud;
        this.longitud = longitud;
        this.hemisferioLongitud = hemisferioLongitud;
    }

    public double calcularDesplazamiento(double lat2, double long2) {  no usages  Santiago Rojas Bonilla
        double dx = this.latitud - lat2;
        double dy = this.longitud - long2;
        return Math.sqrt(dx * dx + dy * dy);

    }

    @Override  Santiago Rojas Bonilla
    public String toString() {
        return "ubicacionEspacial{" +
```

Código Clase PeriodoVisible:

```java
package observatorio;

public class PeriodoVisible {  2 usages  Santiago Rojas Bonilla

        private String periodo;  2 usages
        private ubicacionEspacial ubicacion;  3 usages

        public PeriodoVisible(String periodo, ubicacionEspacial ubicacion) {  no usages  Santiago Rojas Bonilla
            this.periodo = periodo;
            this.ubicacion = ubicacion;
        }
        public ubicacionEspacial getUbicacion() {  no usages  Santiago Rojas Bonilla
            return ubicacion;

        }

        @Override  Santiago Rojas Bonilla
        public String toString() {
            return "periodoVisible{" +
                    "periodo='" + periodo + '\'' +
                    ", ubicacion=" + ubicacion +
                    '}';
        }
    }
```

Resultados métodos de las clases UbicaciónEspacial y PeriodoVisible:

```
jshell>
<press tab again to see all possible completions; total possible completions: 555>
jshell> ubicacionEspacial u= new ubicacionEspacial (10, "norte", 10, "sur");
u ==> ubicacionEspacial{longitud=10.0, latitud=10.0}

jshell> import observatorio.PeriodoVisible;

jshell> PeriodoVisible p = new PeriodoVisible("enero", u);
p ==> periodoVisible{periodo='enero', ubicacion=ubicaci ... gitud=10.0, latitud=10.0}}


jshell>
```

Análisis del código:

importamos la clase de cuerposCelestes y después añadimos la variable marte, donde después pusimos sus datos que fueron su nombre, composición, distancia a la tierra y unidad a la tierra, eso fue para que después por el metodo toString() con solo escribir la variable marte saliera todos sus datos.

Después importamos la clase UbicacionEspacial donde hicimos una variable llamada lugar donde pusimos las cordenadas de la ubicacion, despues llamamos al metodo calcular desplazamiento para que nos diga a cuanta distancia estamos, lo siguiente fue llamar a cada variable para que nos saliera sus datos guardados. Ocupamos /vars para ver lo que habiamos creado

CONCLUSIONES:

En esta actividad pudimos observar que tan importante es la abstracción y el encapsulamiento, por que hubo errores que cometimos al principio desde la actividad 1, y nos dimos cuenta de ciertos datos que nos hacían falta para poder darle solución a los problemas, como identificar que dato nos faltaba declarar dentro de la clase para poder realizar la operación necesaria, nos dimos cuenta como es que si no realizamos de manera efectiva y correcta la abstracción, al querer seguir con el desarrollo de nuestros problemas pues no podremos hacer mucho, porque nuestros "cimientos" o bases en la abstracción no son solidas o claras, y es el principio básico de la programación orientada a objetos.

URl GIT:

https://github.com/soysantiagorb/Actividad2.git

https://github.com/irvingemanuel2003-a11y/actividad2.git